



PRESENTAZIONE RTB

CODERIUS - GRUPPO 22 - A.A. 2025/2026

coderius01@gmail.com

Automated EN 18031 Compliance Verification

Capitolato C1: Automated EN 18031 Compliance Verification

Scopo:

La verifica manuale della conformità alla norma armonizzata RED EN18031 è complessa, soggetta a errori umani e richiede un tracciamento gerarchico dei requisiti.

Soluzione:

WebApp con interfaccia grafica interattiva in grado di importare le componenti di rete del dispositivo, eseguire i Decision Tree e generare i report di conformità offline.

Tecnologie adottate



- **React 19.2.7**
 - Libreria per costruzione di interfacce a componenti
 - DOM virtuale per aggiornare gli elementi specifici che cambiano
 - Ecosistema vasto e maturo
 - Librerie:
 - **React Flow**
 - **React Router DOM**
 - **Fetch API / Axios**
 - **Zod**
- **Strumenti di testing**
 - **Vitest + React Testing Library:** Framework di riferimento per l'esecuzione rapida dei test unitari sui componenti dell'interfaccia.

Alternative considerate



- **Ember.js:** framework monolitico datato che adotta pattern architetturali rigidi, non in linea con i requisiti di sviluppo agile del progetto.



- **Angular:** impone un'architettura articolata che può risultare eccessiva per i requisiti del progetto.



- **Vue.js:** minore disponibilità di librerie di terze parti mature

Tecnologie adottate



- **Python 3.12**
 - Preferenza esplicita della proponente
 - Sintassi semplice e alta leggibilità del codice



Flask

- **Flask 3.1.3:**
 - Backend snello con endpoint minimali e senza database.
 - Librerie:
 - **Flask-CORS:** Dipendenza per abilitare in sicurezza il CORS tra il server Flask e il client React operanti su porte differenti.
- **Strumenti di testing:**
 - **Pytest:** Framework per il testing automatizzato.

Alternative considerate



- **FastAPI:** Funzionalità chiave come I/O asincrono risultano superflue per un backend minimale senza database, introducendo solo complessità.



- **Django:** Framework che comporta un pesante overhead di memoria e moduli rimasti inutilizzati.

Tecnologie adottate



- **Vite**

- Moduli ES Nativi: Serving del codice "on-demand" senza bundling preventivo.
- HMR Istantaneo: Aggiornamento dell'interfaccia in tempo reale al salvataggio.



- **Docker**

- Stesso ambiente di sviluppo per il team.
- Rilascio dell'applicazione sicuro, veloce e affidabile.

Alternative considerate



- **Webpack:** Tempi di avvio e ricompilazione in sviluppo lenti rispetto all'approccio on-demand di Vite.



- **Parcel:** Ottimizzato per Multi-Page Applications tradizionali anziché per Single Page Applications reattive.



- **Podman:** Minore stabilità di Podman Compose rispetto allo standard Docker richiesto da Bluewind.



- **Rancher:** Piattaforma di orchestrazione e gestione cluster sovradimensionata per un'architettura minimale.